



Programovacie jazyky

GADT, moduly, ADT, typy ranku- n , bezbodový štýl, seq, ...

RNDr. Richard Ostertág, PhD. (ostertag@dcs.fmph.uniba.sk)

3. mája 2018

Katedra informatiky, Fakulta matematiky, fyziky a informatiky
Univerzita Komenského, Bratislava

Zovšeobecnené alg. dátové typy

Celočíselné výrazy (konstanty, +, ×)

```
1 data Expr = I Int
2           | Add Expr Expr
3           | Mul Expr Expr
4           deriving Show
```

```
1 exp1 :: Expr
2 exp1 = (I 5 'Add' I 1) 'Mul' I 7
```

`exp1` $\rightsquigarrow$ `Mul (Add (I 5) (I 1)) (I 7)`

```
1 eval :: Expr -> Int
2 eval (I n)      = n
3 eval (Add e1 e2) = eval e1 + eval e2
4 eval (Mul e1 e2) = eval e1 * eval e2
```

`eval exp1` $\rightsquigarrow$ 42

Výrazy rozšírené o porovnanie (<) a bool. hodnoty

```
1 data Expr = I Int | B Bool
2           | Add Expr Expr
3           | Mul Expr Expr
4           | Lt Expr Expr
5           deriving Show
```

```
1 eval :: Expr -> Maybe (Either Int Bool)
```

```
1 exp1 = (I 5 'Add' I 1) 'Lt' I 7
```

`eval exp1` $\rightsquigarrow$ **Just (Right True)**

```
1 exp2 = (I 5 'Add' B True) 'Lt' I 7
```

Syntakticky správne, ale sémanticky nie ($5 + \text{True} < 7$):

`eval exp2` $\rightsquigarrow$ **Nothing**

Funkcia eval pre rozšírené výrazy

```
1 eval :: Expr -> Maybe (Either Int Bool)
2 eval (I n) = return $ Left n
3 eval (B b) = return $ Right b
4 eval (Add e1 e2) = do
5     Left i1 <- eval e1
6     Left i2 <- eval e2
7     return $ Left (i1 + i2)
8 eval (Mul e1 e2) = do
9     Left i1 <- eval e1
10    Left i2 <- eval e2
11    return $ Left (i1 * i2)
12 eval (Lt e1 e2) = do
13    Left i1 <- eval e1
14    Left i2 <- eval e2
15    return $ Right (i1 < i2)
```

- Sémantickú korektnosť musíme kontrolovať ručne.
 - Vďaka monáde Maybe, to nie je až tak neprehľadné.

Fantómové typy (phantom types)

- návrhový vzor, zavedieme
 - zdanlivo nepoužívaný (fantómový) typový parameter a
 - múdre (smart) dátové konštruktory
- spolu znemožnia vytvorenie sémanticky nekorektných výrazov

```
1 data Expr a = I Int
2             | B Bool
3             | Add (Expr a) (Expr a)
4             | Mul (Expr a) (Expr a)
5             | Lt  (Expr a) (Expr a)
6             deriving Show
```

```
1 i :: Int -> Expr Int
2 i = I
```

```
1 b :: Bool -> Expr Bool
2 b = B
```

Fantómové typy (phantom types) – výhody

```
1 add :: Expr Int -> Expr Int -> Expr Int
2 add = Add
```

```
1 mul :: Expr Int -> Expr Int -> Expr Int
2 mul = Mul
```

```
1 exp1 :: Expr Int
2 exp1 = (i 5 'add' i 1) 'mul' i 6
```

Toto už stačí na to, aby sémanticky nekorektný výraz spôsobil syntaktickú chybu:

```
1 exp2 = i 5 'add' b True
2 -- Expected type: Expr Int
3 -- Actual type: Expr Bool
```

Potrebujeme však zakázať štandardné dátové konštruktory a povoliť iba múdre konštruktory (viď. abstraktné DT).

Fantómové typy (phantom types) – problémy

Nevieme definovať múdru verziu Lt:

```
1 lt :: Expr Int -> Expr Int -> Expr Bool
2 lt = Lt
```

- Expected type: Expr Int -> Expr Int -> Expr Bool
- Actual type: Expr Bool -> Expr Bool -> Expr Bool

Nevieme napísať eval:

```
1 eval :: Expr a -> a
2 eval (I n) = n
```

Couldn't match expected type 'a' with actual type 'Int'

Potrebujeme aby priamo dátové konštruktory mohli typ *a* obmedziť (zatiaľ to vedia iba múdre konštruktory):

```
1 exp3 :: Expr a
2 exp3 = I 5
```


Zovšeobecnené algebraické dátové typy

Generalized algebraic datatypes – GADTs

- umožňujú obmedziť návratovú hodnotu dátového konštruktora
 - pri ADT by musel byť **Expr** a
 - pri GADT môže byť napr. **Expr Int** alebo **Expr Bool**

```
1 data Expr a where -- alebo data Expr :: * -> * where
2   I   :: Int  -> Expr Int
3   B   :: Bool -> Expr Bool
4   Add :: Expr Int -> Expr Int -> Expr Int
5   Mul :: Expr Int -> Expr Int -> Expr Int
6   Lt  :: Expr Int -> Expr Int -> Expr Bool
7   -- deriving Show
```

Can't make a derived instance of 'Show (Expr a)'

Zovšeobecnené algebraické dátové typy – pokračovanie

```
1 eval :: Expr a -> a
2 eval (I n) = n
3 eval (B b) = b
4 eval (Add e1 e2) = eval e1 + eval e2
5 eval (Mul e1 e2) = eval e1 * eval e2
6 eval (Lt e1 e2) = eval e1 < eval e2
```

```
1 exp1 :: Expr Bool
2 exp1 = (I 5 'Add' I 1) 'Lt' I 6
```

`eval exp1` $\rightsquigarrow$ **False**

```
1 exp2 = (I 5 'Add' B True) 'Lt' I 6
```

Couldn't match type 'Bool' with 'Int'

Presne určí typ výrazu:

```
1 exp3 :: Expr Int
2 exp3 = I 5 'Mul' I 2
```

Moduly

Moduly – úvod

- modul je zoskupenie vzájomne súvisiacich funkcií, typov, typových tried, konštánt, ...
- program je zoskupenie modulov
 - moduly importujú iné moduly a volajú ich funkcie
 - program sa spúšťa zavolaním hlavnej (main) funkcie
 - z hlavného modulu (Main)
- rozdelenie programu do modulov prináša niekoľko výhod:
 - kontrola menného priestoru (menej kolízií mien)
 - slabo previazané moduly (loosely coupled)
 - väčšia znovupoužiteľnosť
 - ľahšie rozdelenie práce, rýchlejší vývoj
 - ukrytie implementačných detailov (neexportujeme ich)
 - umožňuje vytváranie abstraktných dátových typov (ADT)
 - k ADT viac v nasledujúcej časti

Moduly – syntax

- názvy modulov sú alfanumerické, začínajú veľkým písmenom
- modul s názvom `Meno` je v súbore s názvom `Meno.hs`
- pokiaľ je názvom modulu zložených z viac názvov oddelených bodkou, tak názvy pred bodkou sú adresáre
 - napríklad modul `Data.List` je v súbore `Data/List.hs`
- **module** `Meno` (*{-exportované názvy-}*) **where**
- **module** `Meno` **where** exportuje všetky názvy
- pokiaľ súbor nezačína **module**, tak sa predpokladá:
module `Main`(`main`) **where**
- **module** `Maybe` (**Maybe**(**Nothing**, **Just**), ...) **where**
 - **module** `Maybe` exportuje typový konš. **Maybe** a dátové konštruktory (DK) **Nothing** a **Just**
 - **module** `Maybe` (**Maybe**(. . .)) **where** (exp. $\forall$ DK)
 - **module** `Maybe` (**Maybe**) **where** (neex. žiadny DK)
- moduly, typ. a dát. konš. sú rôzne menné priestory

Moduly – príklad

```
1 module Stack (Stack(Empty,Push), isEmpty, push, pop) where
2
3 data Stack a = Empty | Push a (Stack a)
4
5 instance Show a => Show (Stack a) where -- inštancie sa
6     show Empty = "bottom" -- automaticky všetky exportujú
7     show (Push x s) = show x ++ "|" ++ show s
8
9 isEmpty :: Stack a -> Bool
10 isEmpty Empty = True
11 isEmpty _      = False
12
13 push :: a -> Stack a -> Stack a
14 push x s = Push x s
15
16 pop :: Stack a -> (a, Stack a)
17 pop Empty = stackPanic "Popping from empty stack."
18 pop (Push x s) = (x, s)
19
20 stackPanic :: String -> t -- neexportovaná funkcia
21 stackPanic m = error $ "Stack: " ++ m
```

Moduly – import

```
1 module Main(main) where
2   -- import sa umiestňuje medzi module a prvú funkciu
3   import Stack -- načíta všetky exporty do glob. men. priestoru
4   import Stack (Stack(Empty), push, pop) -- len vymenované
5   import Stack (Stack(..), push, pop)    -- všetky DK pre Stack
6   main = do let s = push 10 (push 20 Empty)
7             let (top, s') = pop s
8             print s      -- 10|20|bottom
9             print top    -- 10
10            print s'     -- 20|bottom
```

- **import Data.List hiding** (nub, sort)
 - naimportuje všetko okrem funkcií **nub** a **sort**
- **import qualified Data.Map** (filter)
 - **Data.Map.filter** je funkcia **filter** z **Data.Map**
 - **filter** je funkcia z Prelude (importuje sa automaticky)
- **import qualified Data.Map as M** (filter)
 - umožňuje namiesto **Data.Map.filter** písať iba **M.filter**
- **import qualified** zabraňuje kolízii názvov (name clash)

Abstraktné dátové typy

Abstraktný dátový typ

- dátový typ spolu s operáciami nad týmto typom, pričom sa ukrývajú implementačné detaily (vrátane štruktúry typu)
- dostupné operácie nazývame rozhranie (interface) ADT
- **Data.Map** a **Data.Set** sú príklady štandardných abstraktných typov

Abstraktný dátový typ – zásobník

```
1 module StackADT (Stack, empty, isEmpty, push, pop) where
2 data Stack a = Empty | Push a (Stack a)
3 -- neexportujeme dátové konštruktory pre Stack
4 instance Show a => Show (Stack a) where
5     show Empty = "bottom"
6     show (Push x s) = show x ++ "|" ++ show s
7 -- exportujeme hodnotu pre prázdny zásobník
8 empty :: Stack a
9 empty = Empty
10 -- inak by sme nevedeli vytvoriť hodnotu typu Stack a
11 isEmpty :: Stack a -> Bool
12 isEmpty Empty = True
13 isEmpty _      = False
14 -- na empty potom môžeme aplikovať push
15 push :: a -> Stack a -> Stack a
16 push x s = Push x s
17
18 pop :: Stack a -> (a, Stack a)
19 pop Empty = error "Stack: Popping from empty stack."
20 pop (Push x s) = (x, s)
```

Abstraktný dátový typ – zásobník – použitie

```
1 module Main(main) where
2
3 import StackADT
4
5 main :: IO ()
6 main = do let s = push 10 (push 20 empty)
7         let (top, s') = pop s
8         print s      -- 10|20|bottom
9         print top    -- 10
10        print s'     -- 20|bottom
```

- v module *Main* nevieme manipulovať s obsahom *s* alebo *s'*
- nevieme vytvárať ani rozoberať hodnoty typu *Stack* a inak ako pomocou rozhrania, ktoré modul *StackADT* exportuje (napríklad: *empty*, *push*, ...)
- ak ostane zachované rozhranie, tak implementáciu môžeme kedykoľvek zmeniť

Typy ranku-n

Typy ranku- n (Rank-N types) – motivácia

Normálne typy v Haskellu sú ranku-1, t.j. $a \rightarrow b \rightarrow a$ je ekvival.:

- `forall a. (a -> forall b. (b -> a))`
 - `forall` môžeme presunúť z pravej strany `->` pred začiatok príslušného operátora `->`, čím dostaneme:
- `forall a b. a -> b -> a`

Majme funkciu f definovanú nasledovne:

```
1 f :: a -> (a, Int)
2 f x = (x, 1)
```

Očakávali by sme, že funkcia `g f x y = (f x, f y)` po zavolaní `g2 f True 'a'` vráti `((True,1), ('a',1))`. V skutočnosti však funkcii `g` Haskell odvodil typ:

```
1 g :: (a -> b) -> a -> a -> (b, b)
```

Typy ranku- n

- predošlý príklad nefungoval, pretože Haskell použil Hindley-Milnerov typový systém pre odvodenie typu funkcie, čiže vie odvodiť iba typy ranku-1
- my však potrebujeme nasledovný typ:
$$\forall b\ c. (\forall a. a \rightarrow (a, \text{Int})) \rightarrow b \rightarrow c \rightarrow ((b, \text{Int}), (c, \text{Int})),$$
ktorý je ranku-2 (forall je na ľavej strane $\rightarrow$), na rozdiel od:
$$\forall a\ b\ c. (a \rightarrow (a, \text{Int})) \rightarrow b \rightarrow c \rightarrow ((b, \text{Int}), (c, \text{Int}))$$
 - čo nie je možné, ak zvolíme a , tak potom už $b = a$ aj $c = a$
- základom pre typy vyššieho ranku je System F (alebo druho-rádový λ -kalkulus)
 - typová inferencia pre System F je nerozhodnuteľná
 - preto pre typy n -tého ranku treba používať typové anotácie
- typ ranku- n je funkcia, ktorá má aspoň jeden argument ranku- $(n-1)$, ale žiadny argument vyššieho ranku

Typy ranku- n – príklad

```
1 f :: a -> (a, Int)
2 f x = (x, 1)
3
4 g :: (forall a. a -> (a, Int)) -> b -> c -> ((b, Int), (c, Int))
5 -- jedine táto typová anotácia je nutná
6 g f x y = (f x, f y)
7
8 v :: ((Bool, Int), (Char, Int))
9 v = g f True 'a'
10 -- ((True,1),('a',1))
```

- iným príkladom pre typy ranku- n je zabezpečenie, aby výstupná hodnota funkcie nevynášala obsah vstupných hodnôt
 - `runST :: (forall s. ST s a) -> a`
- ukážeme si však jednoduchší príklad

Typy ranku- n – príklad

```
1  -- len nemôže vôbec čítať obsah zoznamu
2  len :: forall a. [a] -> Int
3  len = length
4
5  -- total môže pracovať s obsahom položiek zoznamu
6  total :: [Int] -> Int
7  total = sum
8
9  -- funkcia f nie je bezpečná, výsledok môže vynášať obsah
10 f :: forall a. [a] -> ([a] -> Int) -> Int
11 f list h = h list
12
13 --           1                2          binding 2 shadows binding 1
14 g :: forall a. [a] -> (forall a. [a] -> Int) -> Int
15 g list h = h list
16 -- funkcia g je bezpečná, jej výsledok nemôže vynášať obsah
17
18 main = do print $ f [3..10] len      -- 8
19           print $ f [3..10] total    -- 52
20           print $ g [3..10] len      -- 8
21           print $ g [3..10] total    -- typová chyba
```


Bezbodový (point-free) štýl

Bezbodový (point-free) štýl programovania

Známy tiež ako „tacit programming“ alebo „point-less style“ (-:-).

Definícia

Bezbodový štýl, je štýl zápisu funkcie, pri ktorom sa v tele funkcie nespomínajú argumenty (body) v ktorých sa funkcia vyhodnocuje.

Príklady (jasnejší, čistejší, kompaktnejší zápis)

- `sum xs = foldr (+) 0 xs` $\rightsquigarrow$ `sum = foldr (+) 0`
- `sortBy (\x y -> compare (fst x) (fst y))` $\rightsquigarrow$
`sortBy (comparing fst)`
- `f x = g1 (g2 (g3 x))` $\rightsquigarrow$ `f = g1 . g2 . g3`
 - redukuje syntaktický šum (zátvorky)
 - redukuje zavádzanie mien pre nepotrebné premenné
 - bezbodový zápis môže obsahovať viac bodiek :-)

Príklad (nič sa nemá preháňať $\rightsquigarrow$ „point-less style“)

- `g f x y = (f x, f y)` $\rightsquigarrow$ `g = flip =<< (((.).(,)).)`

Bezbodový štýl – pomocníci

- <https://webchat.freenode.net/>
 - Nickname: prezývka
 - Channels: haskell
 - Connect
- /query lambdabot
 - `@pl g f x y = (f x, f y)`
 - `g = flip =<< (((.) . (,)) .)`
 - `@unpl g = flip =<< (((.) . (,)) .)`
 - `g b0 y x0 = ((,)) (b0 y) (b0 x0)`
- <http://pointfree.io/>
 - to isté čo @pl, len priamejší prístup

Príklady zaujímavých kombinátorov

owl: ((.)\$(.))

```
1 owl::(t3->t2->t1)->t3->(t0->t2)->t0->t1
2 owl a b c d = a b (c d)
```

owl (==) 5 (+1) 4 $\rightsquigarrow$ **True**

dot: ((.).(.))

```
1 dot::(t3->t2)->(t1->t0->t3)->t1->t0->t2
2 dot a b c d = a (b c d)
```

(=<<) = join 'dot' fmap

f = (+1) 'dot' (*) = f x y = 1 + (x * y)

Bezbodový (point-free) štýl – príklad ručného odvodzenia

- `any p list = or (map p list)`
 - currying
- `any p list = or ((map p) list)`
 - skladanie funkcií
- `any p list = (or . (map p)) list`
 - `f x = t = f = \x -> t`
- `any p = \list -> (or . (map p)) list`
 - η -redukcia
- `any p = or . (map p)`
 - konverzia z infix na prefix zápis operátora (.)
- `any p = (.) or (map p)`
 - currying
- `any p = ((.) or) (map p)`
 - skladanie funkcií
- `any p = (((.) or) . map) p`
- `any = ((.) or) . map`
 - ľavá sekcia operátorom (.)
- `any = (or .) . map`

Haskell – striktné vyhodnocovanie

Weak head normal form (WHNF)

- výraz je vo WHNF, ak jeho najvonkajšejšia časť je:
 - konštruktor (**True**, **Just** (1+2), (:) (1+2) rest)
 - lambda abstrakcia ($\lambda x \rightarrow$ výraz)
 - neúplná aplikácia vstavanej funkcie ((+) 2, sqrt)

Príklady (výrazov vo WHNF)

- (1 + 1, 2 + 2) – začína konštruktorom (,)
- $\lambda x \rightarrow 1 + 1$ – začína lambda abstrakciou
- 'A' : ("ho" ++ "j") – začína konštruktorom (:)

Príklady (výrazov, ktoré nie sú vo WHNF)

- 1 + 2 – začína úplnou aplikáciou (+)
- ($\lambda x \rightarrow x + 1$) 1 – začína aplikáciou lambda abstrakcie
- "ah" ++ "oj" – začína úplnou aplikáciou (++)

Normal form (NF)

- výraz je v normálnej forme (NF), keď je plne vyhodnotený (žiadny podvýraz už nemôže byť ďalej vyhodnotený)

Príklady (výrazov v NF)

- 10 , $(10, \text{"ahoj"})$, $\backslash x \rightarrow (x + 1)$

Príklady (výrazov, ktoré nie sú v NF)

- $1 + 1$ – môžeme vyhodnotiť na 2
- $(\backslash x \rightarrow x + 1) 1$ – môžeme aplikovať a vyhodnotiť na 2
- $(1 + 1, 2 + 2)$ – môžeme vyhodnotiť podvýrazy $(2, 4)$

Poznámka

- *výraz je NF $\Rightarrow$ výraz je vo WHNF*
- *vyhodnotenie do NF, WHNF môže viesť k pretečeniu zásobníka*
 - $1 + (2 + (3 + 4))$, vyžaduje $2 + (3 + 4)$, čo vyžaduje $3 + 4$

- najzákladnejší spôsob ako zaviesť striktnosť do Haskellu
- funkcia `seq :: a -> b -> b`
 - berie dva argumenty ľubovoľného typu
 - vracia hodnotu druhého argumentu
 - ako vedľajší efekt vyhodnotí prvý argument do WHNF

```
1 foldl' :: (a->b->a) -> a -> [b] -> a
2 foldl' _ z [] = z
3 foldl' f z (x:xs) =
4   let z' = f z x
5   in z' 'seq' foldl' f z' xs
```

```
1 ($!) :: (a -> b) -> a -> b
2 f $! x = x 'seq' f x
```

Prečo foldl' nemusí vždy fungovať

- Nesprávne riešenie:

```
1 f (acc, len) x = (acc + x, len + 1)
2
3 foldl' f (0, 0) [1, 2, 3]
```

- Dôvod: 'seq' sa zastaví na konštruktore (,)

- Správne riešenie:

```
1 f' (acc, len) x =
2   let acc' = acc + x
3       len' = len + 1
4   in acc' `seq` len' `seq` (acc', len')
5
6 foldl' f (0, 0) [1, 2, 3]
```

Vzor s výkričníkom (bang pattern)

- `{-# LANGUAGE BangPatterns #-}`
- `f1 !x = True` – vyhodnotí `x` do WHNF
- `f2 (!x, y) = [x,y]` – vyhodnotí `x`, ale nie `y`
- `f3 !(x,y) = [x,y] = f4 (x,y) = [x,y]`
- `let !z = x` – vyhodnotí `x` do WHNF

```
1 ($!) :: (a -> b) -> a -> b
2 f $! x = let !vx = x in f vx
```

- **import Control.DeepSeq**
 - poskytuje funkcie pre úplné vyhodnotenie dátových štruktúr (NF)
- `(take 10) $! [1..] ~> [1,2,3,4,5,6,7,8,9,10]`
`(take 10) $!! [1..] ~> ⊥`
- `(take 1) $! [1, 2, undefined] ~> [1]`
`(take 1) $!! [1, 2, undefined] ~> undefined`

```
1 import Control.DeepSeq
2 x = [1..10]
```

- `:sp x ~> x = _`
- `x 'seq' 10 ~> 10`
- `:sp x ~> x = 1 : _`
- `x 'deepseq' 10 ~> 10`
- `:sp x ~> x = [1,2,3,4,5,6,7,8,9,10]`

Men. a nemenné datové štruktúry

Meniteľné (mutable) a nemenné (immutable) dátové štruktúry

- po vytvorení už nemôžu byť modifikované
 - môžeme vytvoriť novú štruktúru vychádzajúcu zo starej, ale starú nemôžeme modifikovať (update in-place)
 - nové verzie (modifikácie) nemôžu byť prístupné cez aliasy
- výhody nemenných dátových štruktúr:
 - ľahšie sa o nich uvažuje
 - majú iba jeden stav a v tom ostávajú
 - viac-vláknová bezpečnosť (thread safety), je read-only
 - viac vlákien pracujúcich súčasne ju nemôže poškodiť
 - ľahko sa dajú zdieľať
 - zabráňujú vedľajším efektom
 - keď ich funkcia v tichosti modifikuje
 - stará a nová verzia zdieľajú spoločné časti
 - ľahko sa zisťuje ktorá časť bola v novej verzii modifikovaná (oproti starej verzii)

Nemenné dátové štruktúry – Java, C#, JavaScript, C++

- Java, C#: class String
- JavaScript: immutable.js

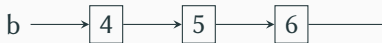
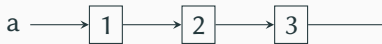
```
1 const { Map } = require('immutable')
2 const map1 = Map({ a: 1, b: 2, c: 3 })
3 const map2 = map1.set('b', 50)
4 map1.get('b') // 2
5 map2.get('b') // 50
```

- C++: Immutable++

```
1 auto a = Array<int>::empty();
2 a = a->push(1);
3 a = a->push(2);
4 auto b = a->push(3);
5 b = b->set(1, 22);
6 for (auto& value : *a) { printf("%d ", value); }
7 // output: 1 2
8 for (auto& value : *b) { printf("%d ", value); }
9 // output: 1 22 3
```

Nemeniteľné dátové štruktúry – Haskell

```
1 a = [1,2,3]
2 b = [4,5,6]
3 c = a ++ b
4 (tail $ tail $ tail c) == b -- True
5 (tail $ tail c) /= b        -- True
```



Meniteľné dátové štruktúry – Haskell

```
1 import Control.Monad.ST (runST)
2 import Data.STRef (newSTRef, modifySTRef', readSTRef)
3 import Data.Foldable (for_)
4
5 sumST :: (Foldable t, Num a) => t a -> a
6 sumST xs = runST $ do
7     n <- newSTRef 0
8     for_ xs $ \x ->
9         modifySTRef' n (+x)
10    readSTRef n
11
12 main :: IO ()
13 main = print $ sumST [1..1000000]
```

Uzávery (closures)

Lexikálny vs. dynamický scoping

- lexikálny scoping – funkcia môže používať iba lexikálne viditeľné premenné v čase definície
 - bez closures – ak by tieto premenné mohli medzi definíciou a volaním zaniknúť, je to chyba
 - s closures – použité premenné $\exists$ v čase definície existujú v uzávere funkcie aj ak by medzitým mali zaniknúť
- dynamický scoping (nepoužíva closures) – premenné sa vždy berú z aktuálneho prostredia (môže obsahovať premenné, ktoré nie sú lexikálne dostupné v čase definície)

Uzávery (closures)

```
1 function Container(priv_value) {  
2   var priv_counter = 2;  
3   function priv_dec () {  
4     if (priv_counter > 0)  
5       { priv_counter -= 1; return true; }  
6     else { return false; }  
7   }  
8   this.getValue = function () {  
9     return priv_dec () ? priv_value : null;  
10  };  
11 }  
12 x = new Container(42);
```

- x.getValue() $\rightsquigarrow$ 42, x.getValue() $\rightsquigarrow$ 42, x.getValue() $\rightsquigarrow$ null
- JavaScript – lexikálny scoping (na úrovni funkcií) a closures

Typová inferencia

- C++:
 - **auto** x = foo(1, 2);
- C#:
 - **var** x = foo(1, 2);
 - **var** myOwnVar = **new** { Name = "John", Age = 100 };
 - vytvorí vlastný typ a inferuje ho
- Java 8:
 - List<String> names = Arrays.asList("Tom", "Dick", "Harry");
 - Collections.sort(names, (first, second) -> first.compareTo(second));